

My report for MVE441

Daniel Peña Fonseca - Dataset A2 - ID: 20031020-T330

June 5, 2026

1 Question 1(a)

To analyze the dataset A2, we first want to visualize the data. We see that for each data, we have 820 values, one for each feature. When plotting data as images, clear insights cannot be extracted; so we plot data as if they were series; i.e. plotting the value as a function of the feature, and realize that this looks more reasonable. The different series look like they have peaks in certain features, so we then want to check if the positions (features) and magnitudes of these peaks are given by the class that they belong to.

For this, we generate a mean series for each class (Fig. 1). We realize that each mean series appears to have the peaks in certain features and of certain magnitude, which may suggest that series of each class are characterized by the magnitude and the position of their peaks. For classification we will only consider the values that series take at the most relevant features for each class. I.e. our classification models will both optimize for the hyperparameter of the model and for how many features it considers per class. To detect the x most relevant peaks from a class, we take the mean series of the class, and select the x features at which the mean series differ the most from the median (the median of the mean series).

The models used are kNN, logistic regression, Random Forest, and LDA (all from the package *sklearn*) using nested CV of $R = 2$ repetitions, $L = 5$ outer folds and $L = 3$ inner folds. To assess the performance of our models, we measure the accuracy and the F1 score, both overall (Fig. 2) and per class (Fig. 3). Results show that with our dimensionality reduction technique (median procedure), all the models are highly able to distinguish between series of different classes. In general, kNN and Random Forest perform better than logistic regression and LDA. This difference in performance suggests that after filtering, classes are highly separable in the feature space (confirmed in section 1c, with Figure 9).

For classes really well separated from others in the feature space, such as class 7 (see Fig. 9), all models perform similarly. Logistic regression can easily separate the class from the rest with multidimensional planes. For LDA, the covariance matrix considers the distribution of data points from all classes; however, for a 'far away' class like class 7, the shape of the distribution is less important, resulting in high performance classification.

For classes with more overlap, such as class 3, Random Forest and kNN perform better. In these cases, the hyperplanes in logistic regression and the Gaussian ellipsoids in LDA cannot properly separate classes. kNN, however, can separate them since it only considers the local neighborhood of each data point, making it more robust for detection of classes that are more spread in the feature space (as far as series from the same class are close from each other). Random Forest yields similar performance, since it is able to divide the space in multiple sections, allowing it to capture both local clustering of data and global structures.

Except for LDA, the rest of the models increase in performance when the sample size increases. This is especially noticeable for kNN: since it uses local neighborhoods for predictions; if the sample size is small, local neighborhoods will not be complete, leading to misclassification due to lack of information.

Note that for this report, data is referred to as 'series'.

1.1 Conclusions

- kNN and Random Forest appear to classify better than logistic regression and LDA. For easily separable classes, simple models such as LDA and logistic regression work; but for overlapping classes, they are not sufficient. In these cases, data points of the same classes tend to cluster in the feature space, making the local model kNN more suitable. Random Forest also performs properly in those cases due to its ability to section the feature space more easily than other models.
- To measure how many features are informative for each model, we optimize for how many features should be considered per class. We do this inside each inner loop of the nested CV to avoid data leakage; and once the features from each class have been selected, a union list is created. Thus, the union list contains all the features that will be used for classification. Figure 4 shows that kNN selects the least amount of total features, followed by logistic regression, Random Forest, and finally LDA. kNN distinguishes properly with few features per class, and it may not be adding more because when adding more (less informative) features, noise is added and distances become less informative. Logistic regression uses more features, since it weights their importance, and less informative features can be coupled with low weighted coefficients. In Random Forest, unimportant features tend to be less considered in splits, so more features can be selected without a decrease in performance. Finally, LDA uses more features, which may be given because it tries to aggregate many weak signals (low informative features) to generate classification ellipsoids that better encapsulate the general distribution of all classes.
- The most noticeable challenge in this data set was to realize that each time series is highly characterized by the position and magnitude of their peaks. It was also challenging to realize that it is sufficient to use the mean series of the different classes to capture the most relevant features for classification. Note that instead of using the mean series, the median series was also considered for this experiment, but classification performance was similar to the mean case, and the mean series was chosen for simplicity.

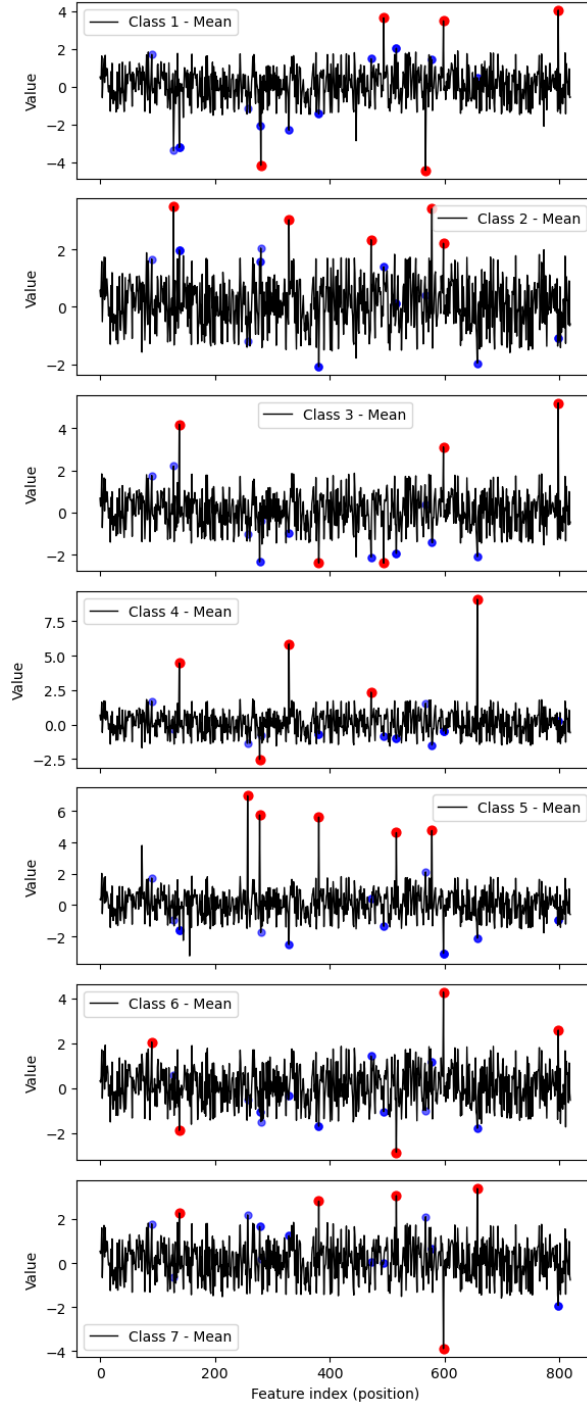


Figure 1: **Mean of each model.** This figure presents the mean series for each class; i.e. for each class, we take all the series from the class, and for each feature, we take the mean of all the series. Instead of the mean, the median could also be used to increase robustness, however, results became more noisy and performance was not improved, so the mean was finally chosen for simplicity. Visualizing the different mean series, classes appear to be mainly characterized by the features at which their peaks appear. To capture the x most prominent peaks from series of each class, we search for the x points that are more deviated from the median of the mean series. For $x = 5$, we obtain the plots that are presented in this figure. Red points show the 5 most prominent peaks from the class under study, while blue points appear in features where other classes have their most prominent peaks.

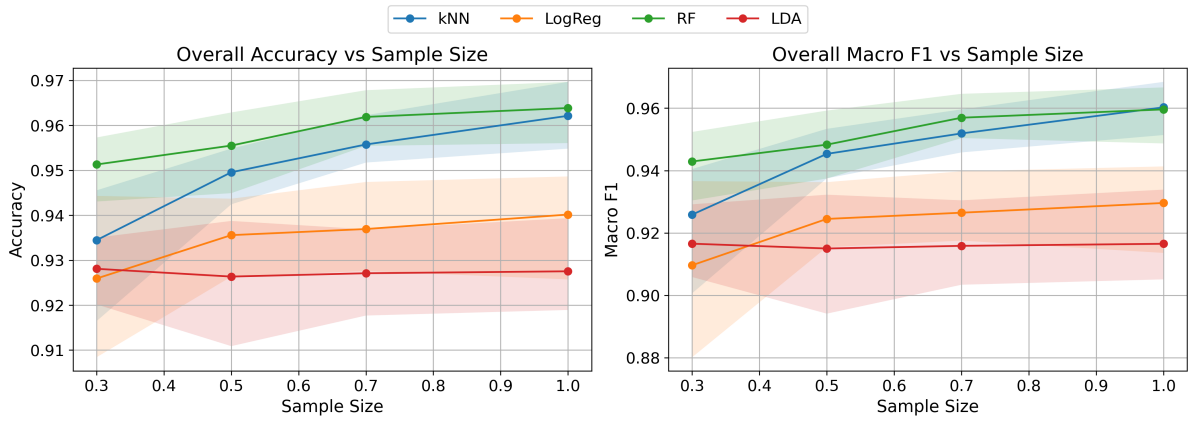


Figure 2: **Overall performance of methods.** This figure presents the overall accuracy (left) and the macro F1 score (right). In the x-axis we present the percentage of the X_TR dataset that is used for the study (30%, 50%, 70% and 100%). For each model and each sample size, we represent the average performance with a colored circle; and the minimum and maximum performance with the shaded areas. This criterion is used for all the report. On average, the Random Forest model performs better than the rest across all sample sizes, followed by kNN, then logistic regression, and finally LDA. Note, however, that in terms of F1 score, kNN reaches the average performance of Random Forest when we use the complete dataset for training.

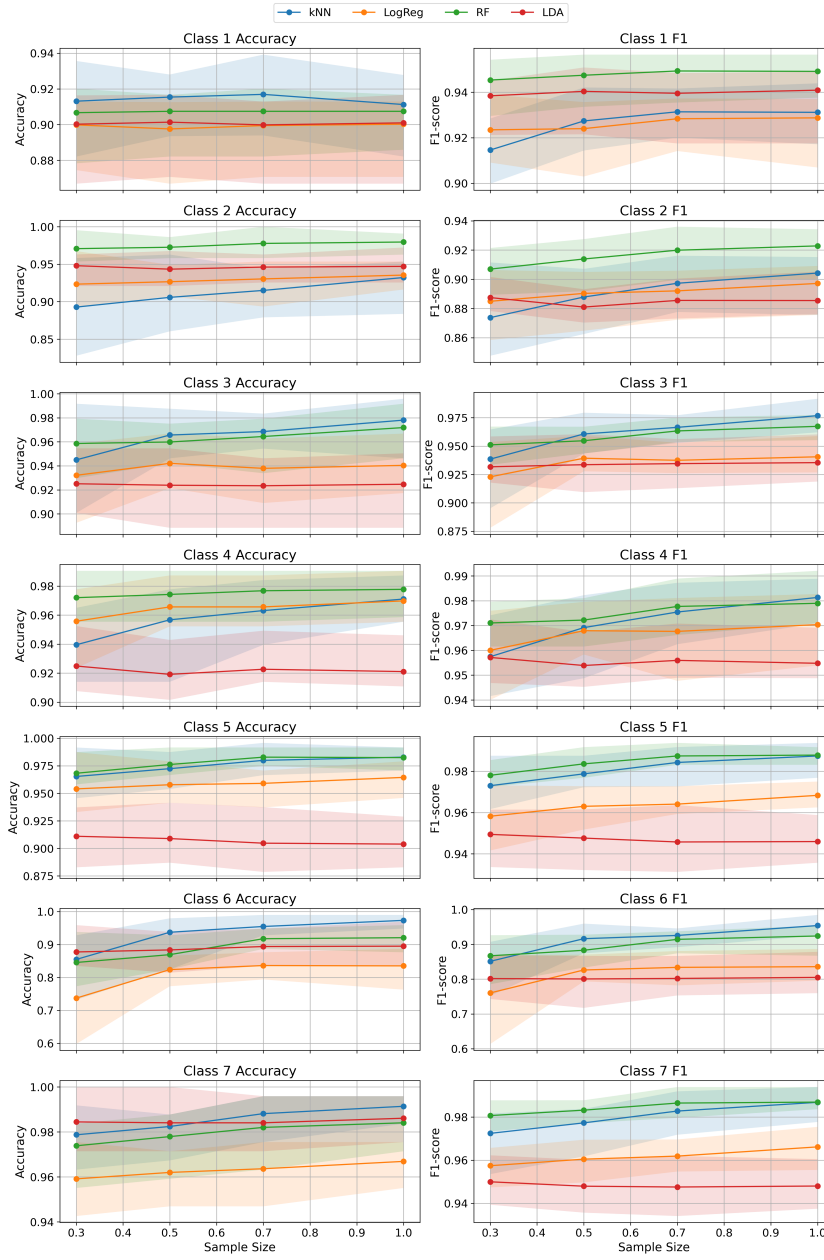


Figure 3: **Class-specific performance of methods.** This figure presents the class-specific accuracies (left column) and the class-specific F1 scores (right column); where each row shows the results for a certain class. In the x-axis we present the percentage of the X-TR dataset that is used for the study (30%, 50%, 70% and 100%). Results are obtained taking into account the $R \times K$ runs performed for each sample size. Results show how, in general, kNN and Random Forest perform better than logistic regression and LDA. Data from class 6 presents plenty of overlap with other classes (as later confirmed in section 1c with Fig. 9), which explains why the performance is worse for this class than for the rest. This class also has less data than other classes, which makes it be less considered, especially by Random Forest, where rules of decision trees favor classification of majority classes; and LDA, where the shape of ellipsoids will resemble other classes' data distributions more than class 6 distribution.

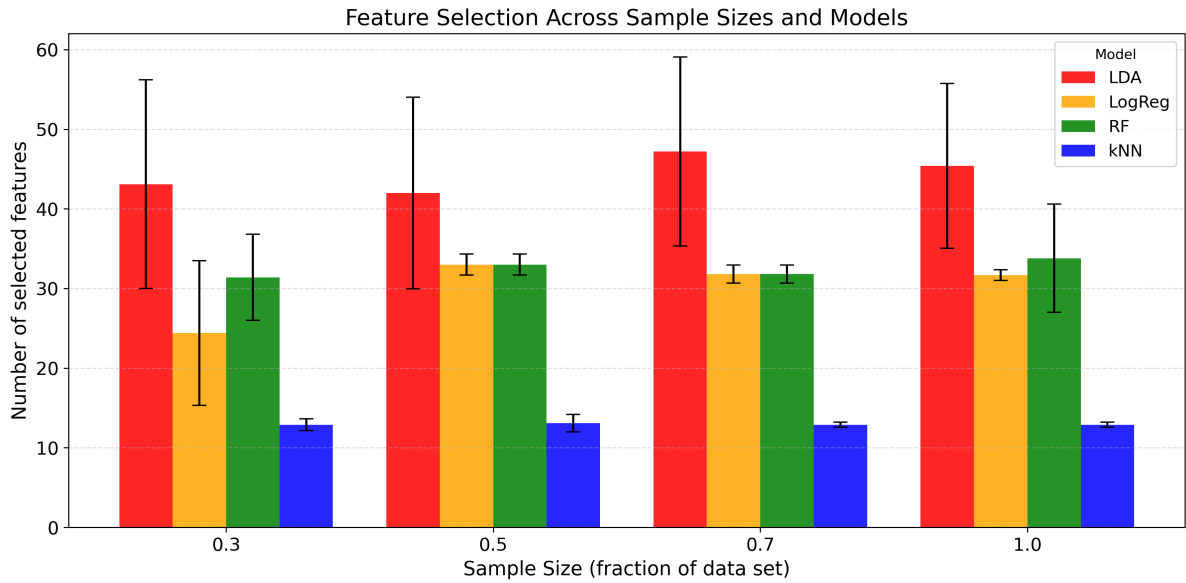


Figure 4: **Number of informative features.** This figure presents, for each sample size, the average number of total features that each model uses for classification. In the x-axis we present the percentage of the X-TR dataset that is used for the study (30%, 50%, 70% and 100%). For each sample size, 4 bars are drawn (one per model), where each bars height indicates the average number of pixels that the model uses for classification. An error bar is included to account for the standard deviation of the $R \times K$ runs. This criterion for the error bars is only used in this figure in the whole report, to make the visualization easier. Note that kNN is the model that uses the least amount of features for classification, followed by logistic regression, Random Forest, and finally LDA.

2 Question 1(b)

Context and Motivation - What is special about our data?

In the previous section, we saw that series from different classes appear to be mainly characterized by their peaks (Figure 1). Both by 'where these peaks appear' (in which features), and by the magnitude of the peaks. Since we are searching for a disruption technique; i.e. a technique that makes our models lose performance, we should take this behavior into account. If series are mainly characterized by the values they take at certain features, ignoring the most useful features will lead to a significant decrease in performance.

Procedure

For this, we follow the same approach as in Question 1(a): we use the 4 classification models for different sample sizes, and in each of these we use nested CV to find the optimal hyperparameters of the model and the optimal number of features that we should consider from each class.

In Question 1(a), we created one list of features per class, and then, a union ($\cup$) list was computed. In this case, however, we want to also capture which of the features from the union list are the most useful; i.e. which of these were selected by more than one class. These features will indicate at which 'positions' multiple classes have their peaks located. To capture these features, inside each inner loop of the nested CV, we sort the union list in a way that the more useful a feature, the closer it is to the beginning of the list.

E.g. imagine that we have 3 classes, and each selects 2 features. Class 1 selects features 5 and 6; class 2 also selects 5 and 6; and class 3 selects 6 and 7. Here feature 6 is selected three times, 5 is selected twice, and 7 is selected once. Thus, our list will be: [6,5,7].

Since we want to avoid using the most useful features, we will ignore the first 25% of the features from the sorted union list (i.e. we keep the worse 75% of the selected features), and evaluate the performance of our models (Fig. 6). We also evaluated the models when ignoring the first 50% to observe an even more significant drop in performance (Fig. 6).

Since this procedure is performed inside each inner loop of the nested CV, we are already rerunning multiple times to make sure that our assessment is stable.

This procedure is equivalent to a 'disruption of the data', since by not considering some features of the data, we are essentially deleting the information that those features of the data possess, and therefore disrupting the data.

Results

Results (Fig. 6 and 7) show that kNN is the most resilient model, followed by Random Forest, Logistic regression, and finally LDA. Note that when using the worse 75% of the selected features (Fig. 6), there is already a noticeable decrease in performance if we compare it to the results we obtained when using the complete list (Fig. 3). However, when using the worse 50% of the list (Fig. 7), the performance with respect to the 75% case drops significantly more than it dropped from the 100% case (Fig. 3) to the 75% case. This especially applies for non-kNN models.

While most of the models suffer a noticeable drop in performance, the kNN model remains robust, maintaining a high accuracy for nearly all classes. This might be given because the kNN model tends to select few features per class; i.e. it yields better performance when it only selects the features where the most prominent peaks appear for each class (around 2 to 3 per class), see Fig. 4 and 12. For visualization, we select the top 3 features per class over the entire dataset, and see, from the union list, how many classes select each feature (Fig. 8). Few features appear to be selected by more than one class, which explains why the kNN is still really robust in the 75% case.

2.1 Conclusions

- When analyzing how the performance of the models evolve with the sample size, no meaningful conclusions can be extracted. For some classes, the performance decreases when increasing the sample sizes, while for others the accuracy decreases. Also, with the error bars being so large, it is hard to draw any conclusions.
- Across all classes, kNN is the most robust model, since performance barely decreases from the 100% case to the 75% case. However, all models suffer noticeably when only the bottom half of the union list is preserved.
- The decrease in performance from all models is expected: our dimensionality reduction technique relies solely in the position and magnitude of the peaks from series; thus, ignoring the features that characterize the most amount of classes, will surely lead to a drop in performance.
- However, some interesting insights can be extracted from visualizing Fig. 7. Even though the top half of the features are ignored now, the classes that were more separable in the 100% case, such as class 5 and class 7 (see Fig. 9) still yield high performance. This behavior may indicate that the position of their peaks is not shared with many other classes, and therefore most of their relevant features are still being considered in this 50% case. This is reasonable, since the fact that features are not shared with other classes might explain why these classes present high separability in Fig. 9. The high separability could also be because features are shared with other classes, but the magnitude of the peaks were noticeably different from those in the rest; however, these results (Fig. 7) point towards a difference in feature selection by these classes.

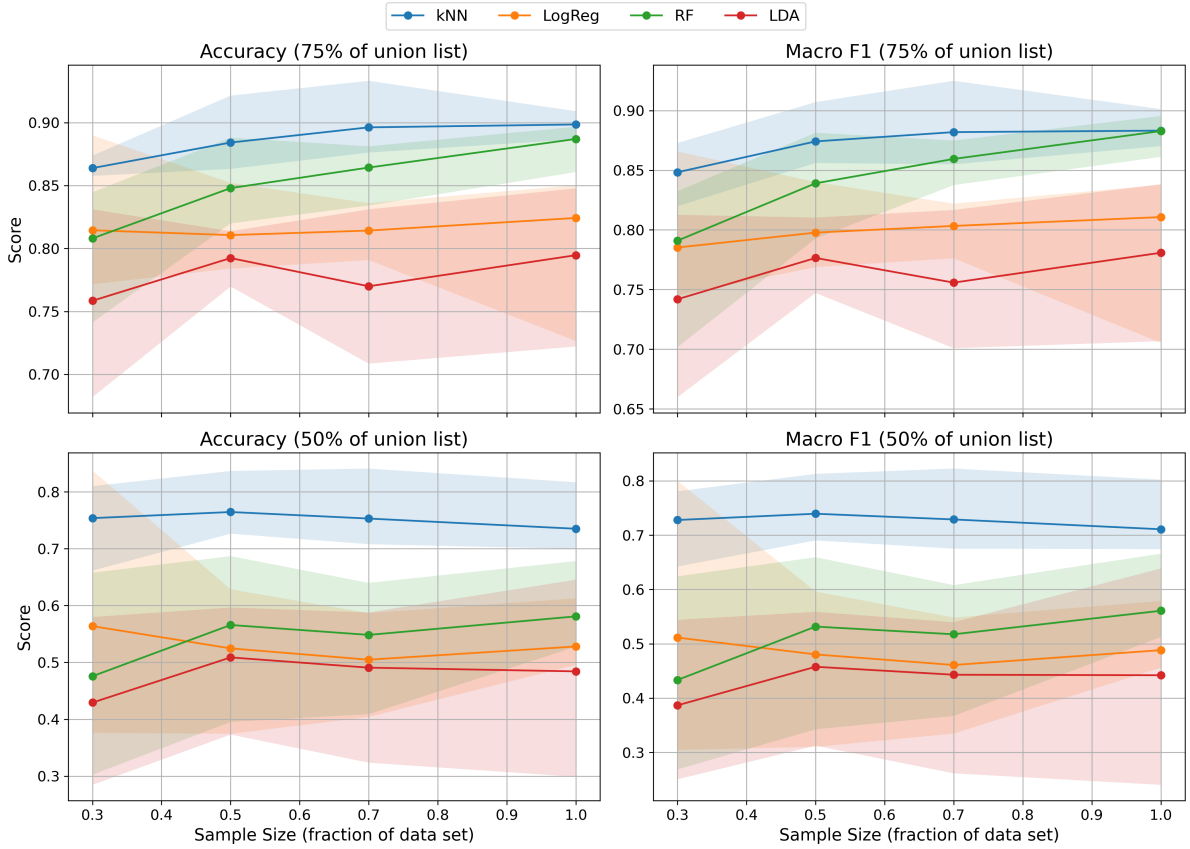


Figure 5: **Overall accuracy and overall F1.** This figure presents the results obtained when using only the lower 75% of the feature union list (top row) and when using only the lower half of the list (bottom row). The classifier ranking is still similar to the one we had observed previously (Fig. 2), although with a noticeable drop in performance. This shows that models are highly reliable on features where peaks appear for more than one classes, since these are the ones ignored with this approach. Note, however, that kNN still maintains a high performance even when features are extracted from the feature list. This is explained by Fig. 8, since it shows that for the kNN common feature selection, there are very few features selected by multiple classes, making the kNN model more robust to this disruption technique.

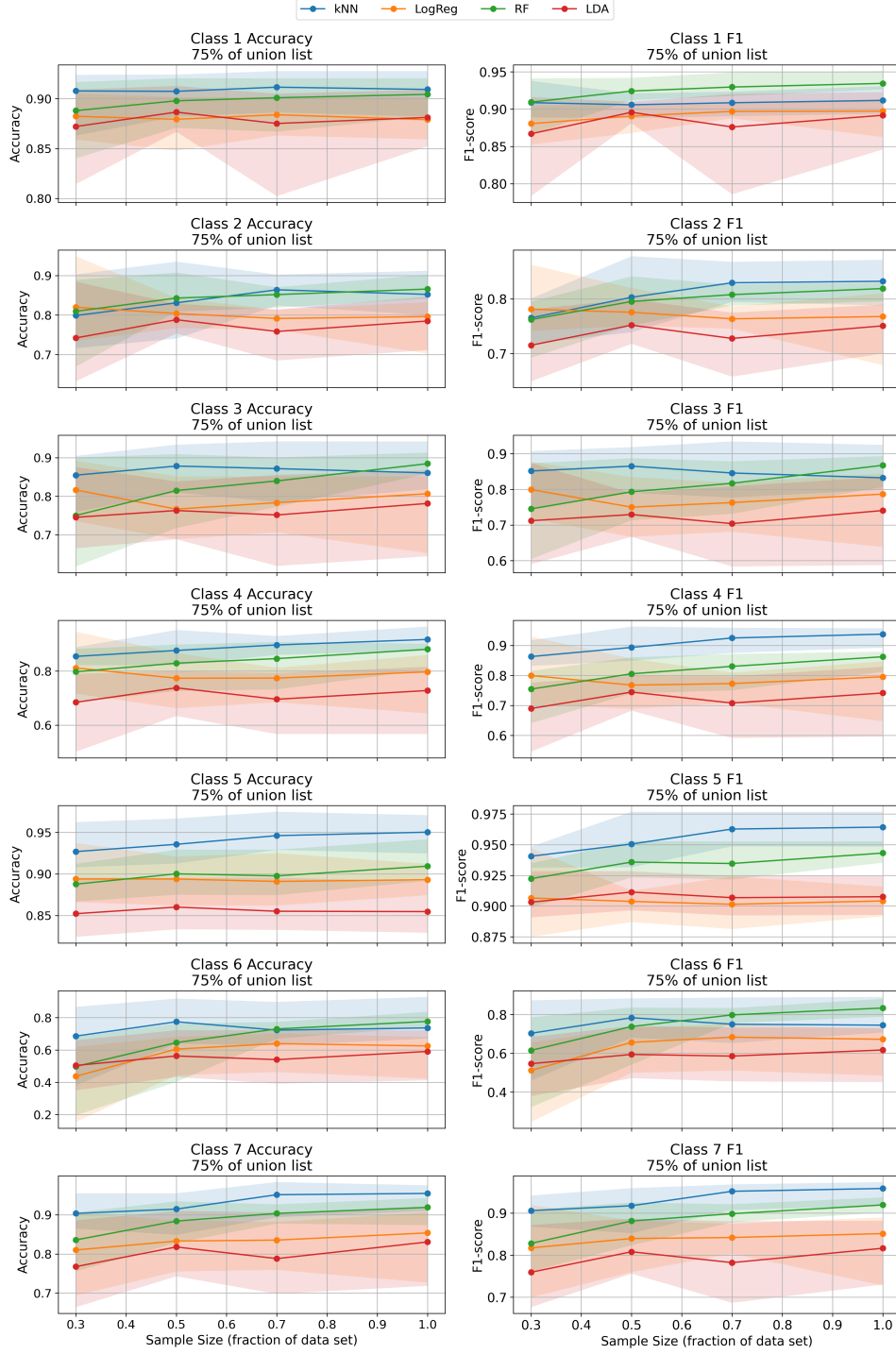


Figure 6: **Class-specific performance - 75% case.** This figure presents the per-class accuracy (left row) and f1-score (right row) when we only use the low 75% of the feature union list. Some decrease in performance appears for most of the models, especially for class 6. This might suggest that class 6 (overlapping with some classes in Fig. 9) shares important features with other classes, making its classification suffer noticeably with this disruption technique.

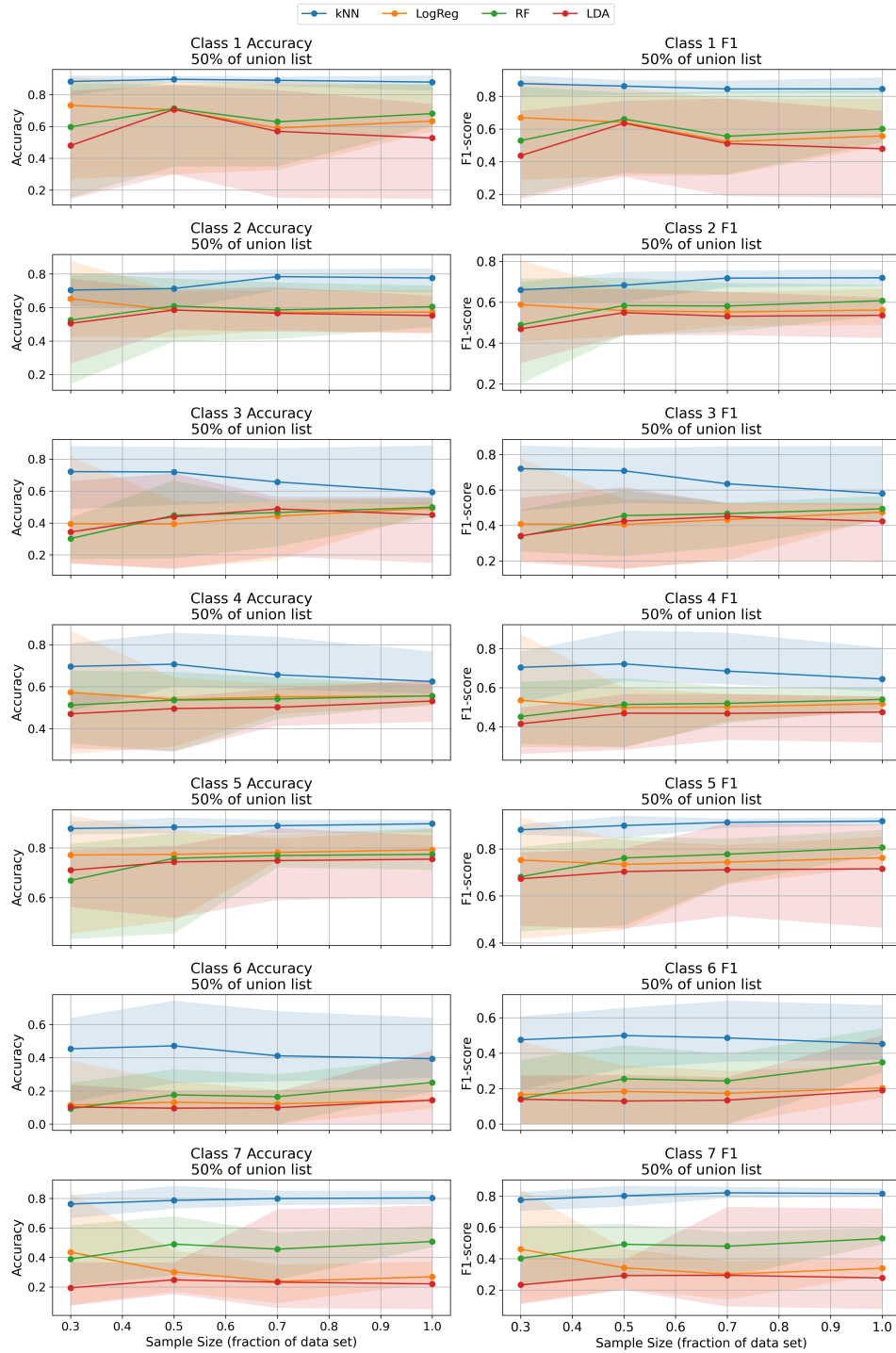


Figure 7: **Class-specific performance - 50% case.** This figure presents the per-class accuracy (left row) and f1-score (right row) when we only use the low 50% of the feature union list. The drop in performance from the 75% case (Fig. 6) to the 50% case is noticeably bigger than the one suffered from the 100% (Fig. 3) case to the 75% case. Note also that the kNN model is still robust in this case, given by the fact that, as we can see in Fig. 8, few features are shared by multiple classes, making the model highly robust to this disruption technique.

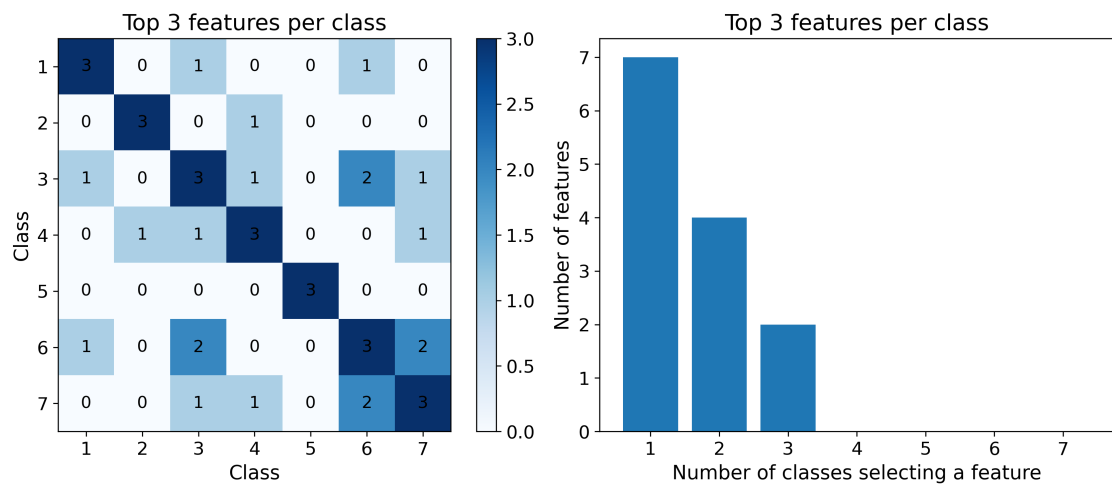


Figure 8: **Usefulness of selected features.** The kNN model tends to select around 3 features per class, as later presented in Fig. 12. When selecting the 3 most useful features per class for classification, a union list is created with these. This figure presents information from the features of the union list; particularly by which classes and by how many classes are these features selected. This way, we can visualize how useful are the features, and most importantly, observe that there are few features selected by multiple classes. Only 2 features of the union list are selected by three classes, 4 selected by two classes, and 7 selected by just one class. This behavior is crucial to understand why the kNN model is so robust in the case of ignoring the top 25% features (75% case): since the top features are not selected by multiple classes, ignoring the top 25% of features will not be as relevant as to decrease performance noticeably.

3 Question 1(c)

In this case, we want to detect evidence of mislabeling in our dataset, and the model that we use for mislabeling detection is kNN.

First, we want to visualize the data distribution in two dimensions. Using all the features would, a priori, not be really useful, since we have seen that series are mainly characterized by the values they take at certain features. For that reason, we first do an initial dimensionality reduction by considering only the top 5 features of each class (median procedure), and then we generate the union list. Once the final features have been stored, we apply PCA, followed by t-SNE. UMAP was also applied. Results for both t-SNE and UMAP are presented in Figure 9.

Note that these plots show how classes are in general, well separated; however, there are some exceptions that can be visualized by analyzing the bottom row of Figure 9.

- Class 1 and class 2 appear to overlap highly in the 2D projections. This could indicate that multiple data labeled as class 1 are mislabeled and actually belong to class 2. But it could also show that certain class 2 series have similar properties to class 1 series; i.e. that peaks appear in the same features and with similar magnitudes.
- Class 6 also seem to partially overlap with different classes, especially with class 3. This could indicate mislabeling, but it may also occur because in the feature space formed by the selected features, series from these classes lie close. The latter is supported by Figure 1, where one can see how two of the most important peaks from both classes lie in the same features and with similar magnitudes.

Once the data distribution has been visualized using these dimensionality reduction methods, we want to use our kNN model, in this case using 10 nearest neighbors, to detect suspicious data points. Using the kNN model, we flag the data points for which the class predicted by our model does not match its label. In the top row of Figure 10, the flagged data points are marked with black circles. With this method, a total of 349 data points were flagged, representing 4% of the entire dataset.

For each data marked as 'suspicious' by the model, we save the model's prediction and the label. With these results, the matrices presented in the bottom row of Figure 10 are obtained. Note that the two clear cases of overlap that we discussed (class 1 with class 2, and class 6 with class 3) are captured by these matrices. Here, multiple series labeled as class 2 are labeled as class 1 by the model. The same behavior appears between class 3 and class 6, to a smaller extent.

Although these results seem reasonable, we need to verify that this technique for mislabeling detection works. To verify this, we randomly select 500 points of the dataset and change their label, with the new label being randomly selected among the others. With this procedure, we want to check if our model flags these as 'suspicious', which would indicate that our model is capable of detecting mislabel observations.

Figure 11 shows the data points artificially mislabeled as black crosses, and red circles are used to surround the black crosses that our model flags as 'suspicious'. Both for the t-SNE and for the UMAP projections, our model flags 99% of the artificially mislabeled observations, showing the ability of the model to detect mislabeling.

However, the high accuracy that our model presents for detecting mislabeled observations does not give information about whether the model gives false positives. Therefore, we cannot ensure that the 349 data points flagged as 'suspicious' are actually mislabeled, but rather say that if there are some mislabeling in the data set, we are likely capturing them with these flags.

3.1 Conclusions

- To detect mislabels, we first visualize our data. For this, we select the top 5 features per class (using the median procedure), create a union list, and apply PCA, followed by t-SNE. The UMAP projection is also presented. With these projections, a noticeable overlap between class 1 and class 2 is visually detected; as well as a less significant overlap between class 3 and class 6.
- Note that projections in Fig. 9 have been obtained by selecting 5 features per class, which limits the usefulness of the plot. The kNN model, for example, tends to select only 3 features per class, as one can see in Fig. 12; and in that case, classes could be more (or less) separated than what is shown in the 2D projection from Fig. 9.
- Nevertheless, with this selection of 5 features per class, we use a kNN model with the parameter k fixed at 10 to detect mislabeling. With this approach, we flag as 'suspicious' data points for which the model's prediction does not match their label. 349 data were considered 'suspicious' (Fig. 10).
- To verify that our model for mislabeling detection works, we introduce 500 mislabeling observations and see if our model is able to detect them. 99% of these observations are flagged as suspicious by our model, indicating high accuracy of mislabeling detection (Fig. 11).
- However, we should take into account that we cannot ensure that among the 349 data points flagged by our model, there are no false positives. With the high accuracy obtained for detection of artificial mislabeling, we can only state that if mislabeling observations are present in the data set, we are (with high probability) capturing the majority of them with our model.

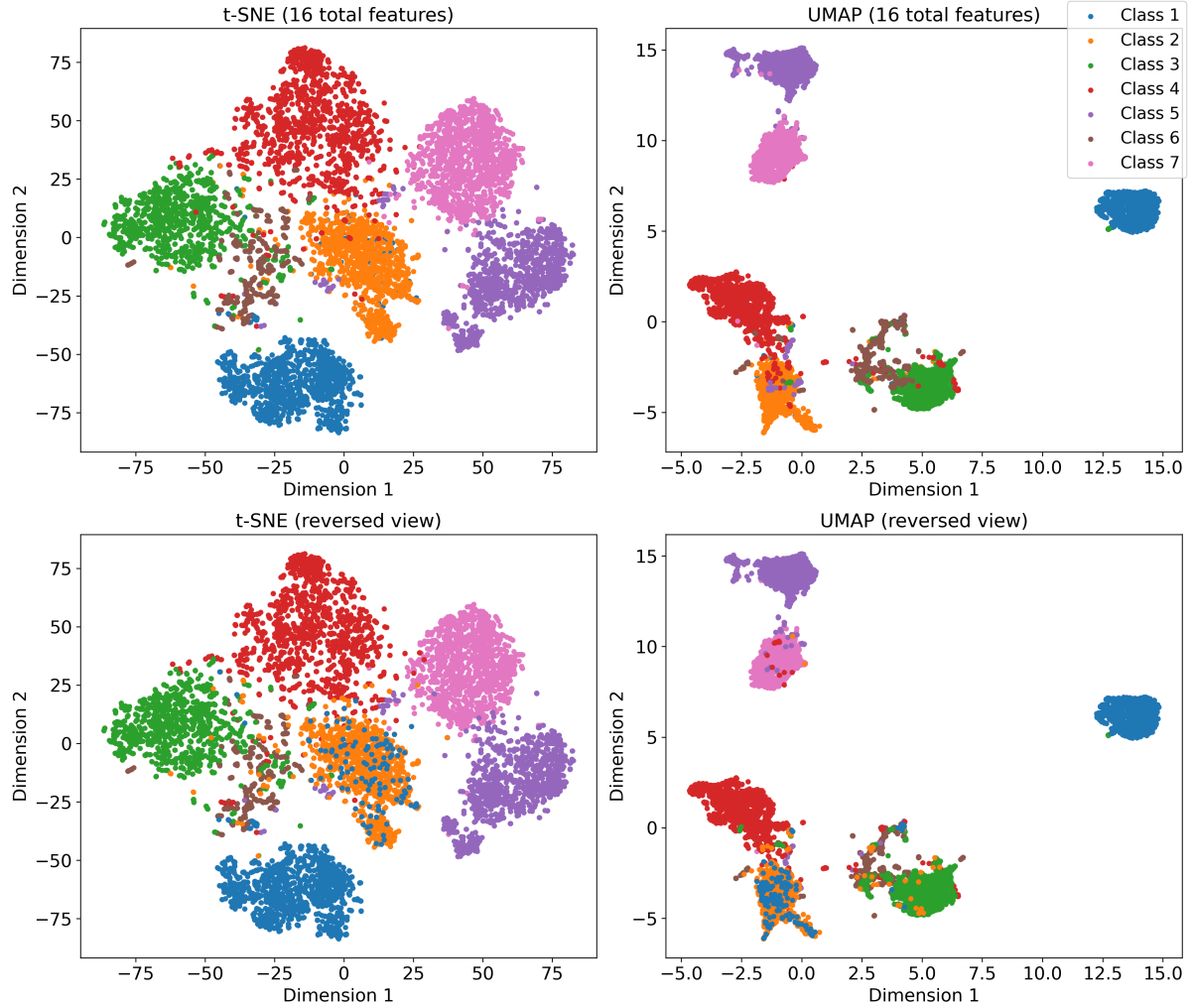


Figure 9: t-SNE and UMAP projections. This figure presents the t-SNE projection (left) and the UMAP (right) for all the dataset. To obtain these results, an initial filtering was performed using the median-procedure, selecting the 5 most relevant features for each class. When projecting over 2 dimensions to make the distribution visible, there is noticeable separation between classes. Note that the bottom row plots show how class 2 and class 1 overlap highly. Some overlap is also present between class 6 and class 3. The top row plots were generated plotting the data points in class order (class 1, then class 2, and so on); while the bottom row plots were generated plotting the data points in reverse order (class 7, class 6, and so on); this way, the bottom row shows class overlap that was not visible in the plots from the upper row.

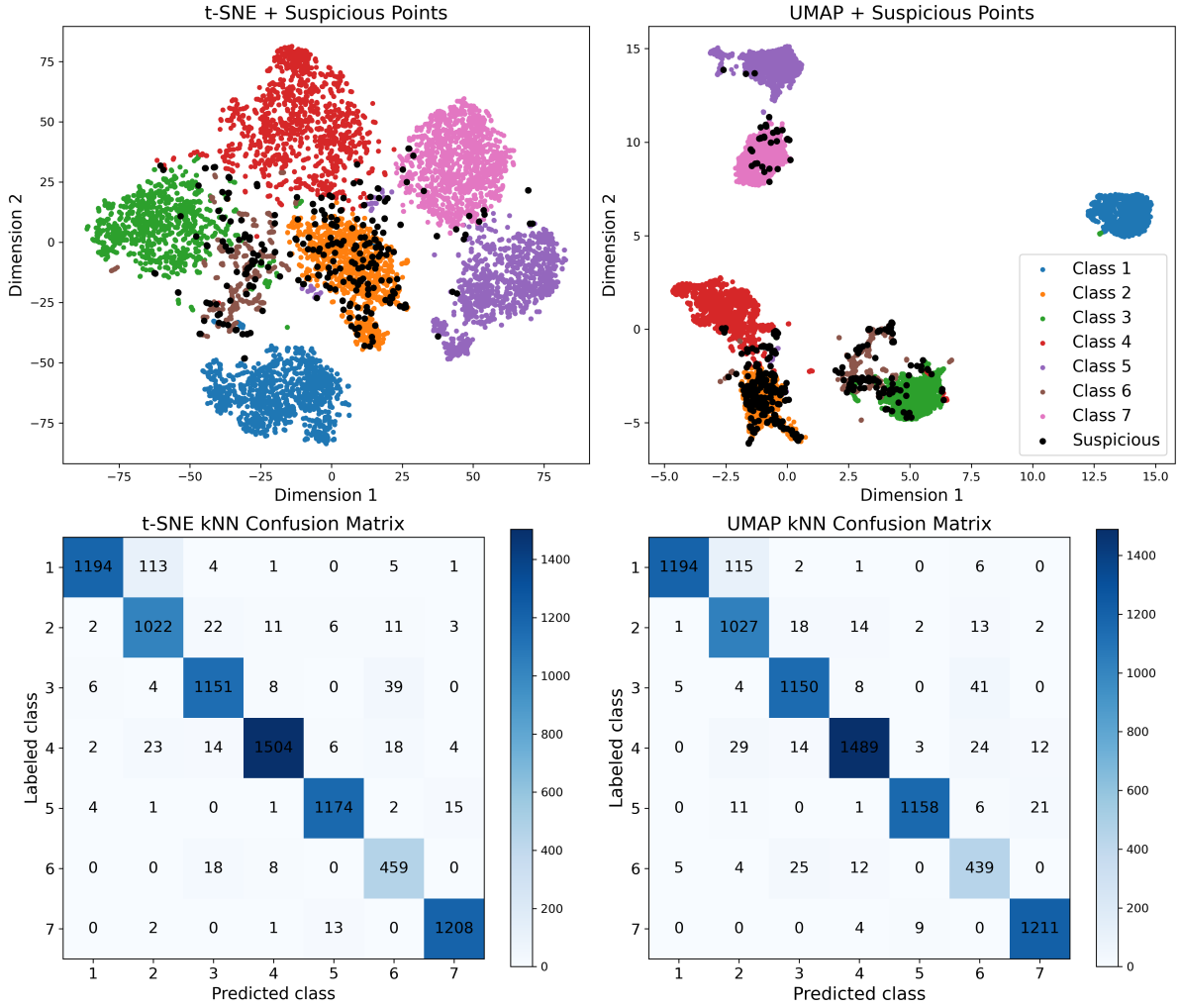


Figure 10: **Potentially mislabeled observations.** This figure presents the t-SNE projection (upper left) and the UMAP (upper right) for all the dataset, with black circles indicating the data points labeled as 'suspicious' by our mislabeling-detection model. To create the 'confusion' matrices, we use the data points that our model marks as 'suspicious'; i.e. the data points that our mislabeling-detection model classifies as a certain class, but they have a different label. Note that the overlap of data points from class 1 and from class 2 mentioned previously is reflected in the confusion matrix, where many data points labeled as class 1, are predicted as class 2 by the kNN model. Some overlap between class 3 and class 6 is also reflected in the confusion matrix.

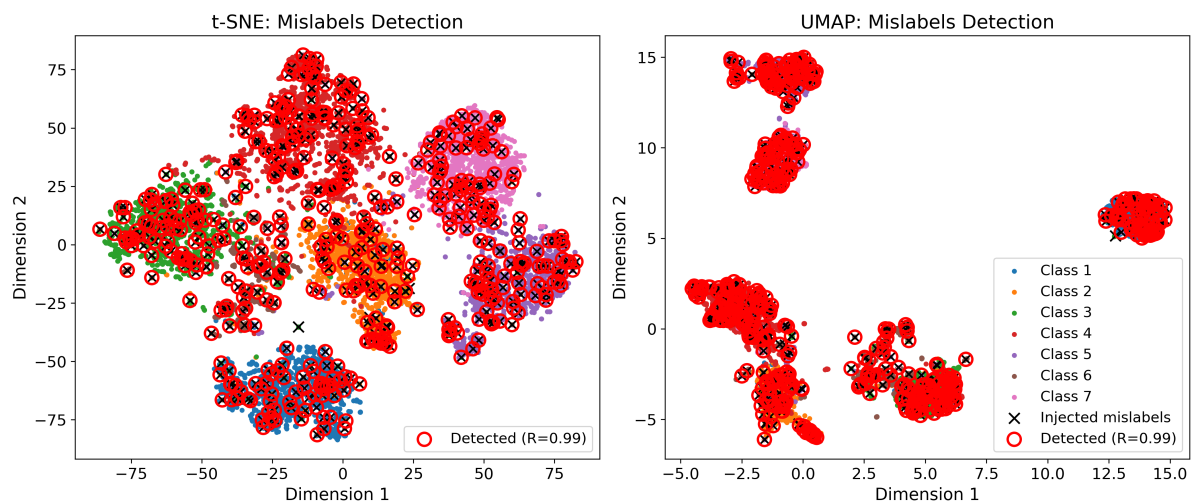


Figure 11: **Artificial mislabeling.** This figure presents the t-SNE projection (left) and UMAP projection (right). In each case, we select 500 data points (black crosses), and we change their label; we then apply our mislabeling-detection model to see how many of these artificially mislabeled data points are detected (red circles). In both cases, our model flags 99% of the data points that suffered manual mislabeling as suspicious, which shows the ability of our model to detect mislabeled observations.

4 Question 1(d)

In this case we will use our two best performing models from Question 1(a). In that question, we saw that, on average, kNN and Random Forest were the most reliable classifiers. For this task, however, we will not perform a grid search again to optimize for the hyperparameters and the number of features that should be selected per class. Instead, we observe which were the most selected parameters in Question 1(a), and use those for this task. Results are presented in Fig. 12. Given these results, for the kNN model we will choose $k = 3$ with 3 features per class; and for the Random Forest we will use 500 estimators, each of them with a maximum depth of 20 levels, and choosing 10 features per class. Note that with this procedure we are not ensuring that the combinations of hyperparameters with these number of features is are the best; just that, independently, these parameters are the most selected ones. However, given that they are noticeably more selected than the rest of parameters, these combinations will likely be, in most cases, the best one.

Once the models have been selected, we will compare how the confidence and uncertainty differ from the out-of-fold test sets of the training data set (X_TR) to the test data set (X_TE). Note that the evaluation is performed out of sample in the first case to avoid data leakage.

The confidence is defined as follows. For a certain series, our models assign different probabilities: the probability that it belongs to class 1, to class 2, etc.; and the class with the highest probability will be the prediction. This highest probability is the confidence of prediction for the series. A model with high confidence will be more 'sure' about a classification than one with less confidence.

To consider what is the general confidence of a model, we will compute the confidence that the model has had when classifying the different series out-of-fold. In the case of kNN, results show that for most of the series, the model is highly confident about its prediction (top left in Fig. 13); while Random Forest presents a lower confidence in prediction. For kNN we find some data for which the confidence is lower. These will most likely correspond to data from class 2 (because it overlaps with class 1), and from class 6 (because it overlaps with multiple classes).

The other metric used is the uncertainty or entropy. For a certain data d , recall that each model computes the probability that it belongs to class 1 (p_1), to class 2 (p_2) and so on. The classification uncertainty is defined as $H(d) = \sum_i p_i \log(p_i)$. A model with perfect confidence has 0 uncertainty, while a model that assigns a similar probability to all the classes presents a higher classification uncertainty.

The bottom row of Fig. 13 presents the uncertainty distribution for the kNN model (left) and for the Random Forest (right). The major insight from this study is that, for the kNN model, the data that were classified with lower confidence appear to have the same uncertainty, which may indicate that they belong to the same class or the same region of the feature space, and the model is assigning similar probabilities when classifying data in the region. This insight contributes to the hypothesis that these data might belong to class 2 or class 6.

Also note that the confidence and uncertainty distributions for the out-of-fold sets and for the test data set are really similar. This might indicate that the data points from X_TE belong to the same distribution as those in the original data set X_TR.

Finally, we study the drift in the test data set and in the out-of-fold sets. For this, we use the Kolmogorow-Smirnov (KS) test, which measures how different are the cumulative probability distributions, of two sets. For this, we will analyze the KS statistic between the training folds and the out-of-fold sets, and the KS statistic between the training set (X_TR) and the test set (X_TE). A KS value of 0 would appear if the distributions were identical, whereas a KS value of 1 denotes that there is no overlap in the distributions.

Results for the test data set, X_TE, show that with both the feature selection from kNN and from Random Forest, there is nearly no drift (Fig. 15). With these findings, we would expect a classification for X_TE similar to that obtained in Question 1(a) when using the entire X_TR.

4.1 Conclusions

- For this question, we have used the kNN model and the Random Forest. To select the hyperparameters and the number of features per class for the models, we have visualized which were the most selected ones in Question 1(a) when we optimized (Fig. 12).
- By visualizing the confidence and uncertainty distributions (Fig. 13), we see that the kNN model is more confident than the Random Forest, and that the kNN model appears to have certain regions in feature space that are more complicated to classify. We can also see in Fig. 14 that, when predicting a certain class, kNN presents high confidence for all classes. On the other hand, the Random Forest has different confidence depending on the class. For example, the Random Forest loses confidence for predictions of class 6, which is the minority class (see Fig. 16).
- Finally, we study the drift using the KS statistic, and KS values of around 0.02 are obtained when comparing the test data set, X_TE, to the training data set, X_TR. These values, obtained both for the feature selection of the kNN model and the Random Forest, suggest that there is nearly no drift. Furthermore, the small drift (around 0.02) that we find is also present in the comparison between the training data from the folds, and the out-of-fold test sets. These two groups, however, should have 0 KS value, since they both belong to the same distribution. Since the same KS value is being obtained when comparing across the training data set (in vs out of fold) and when comparing with the test set (X_TR vs X_TE), we conclude that this KS value does not indicate that drift is present, but rather is a product of the finite sample size of our data.
- With these results, we can conclude that with high probability, the data from the test data set, X_TE, belong to the same distribution as the training data, X_TR, with no drift present.

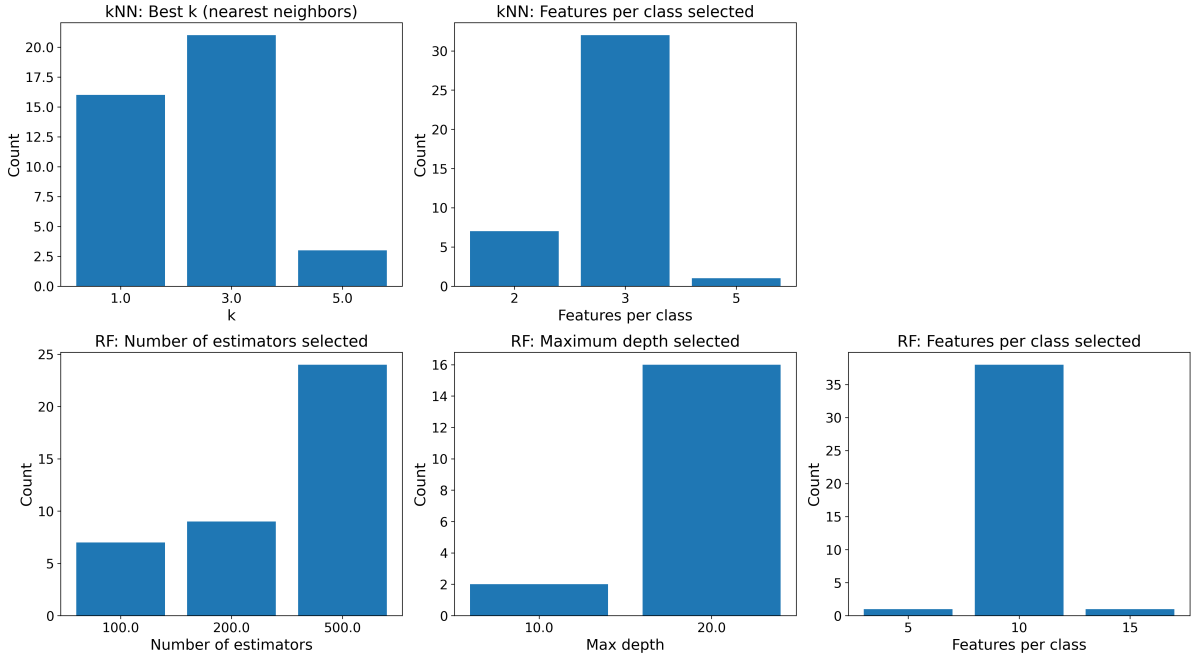


Figure 12: **Best parameters from kNN and Random Forest models from Question 1(a).** This figure studies which were the hyperparameters that our models kNN (top row) and Random Forest (bottom row) selected when they were optimized in Question 1(a); i.e. when doing grid search, how many times was each element of the grid selected as the best. Since we want to take the best performing models for task 1(d), we see that for kNN, the hyperparameter $k = 3$ works best, and we will combine it with selecting 3 features per class. For Random Forest, we will use 500 estimators, with a maximum depth of 20 levels, and we will select 10 features per class. Note that we are not measuring if the combinations between any two parameters works properly, just that they separately are selected the most amount of times. However, given the amount of times that the parameters that will be used were selected, the combination will likely be the best. Also note that in some cases (such as the number of estimators of the Random Forest) we are in the limit of the grid search, which suggests that more hyperparameters should be tried. However, due to time constraints and computational cost, no more parameters were considered. For the maximum depth, no maximum depth was tried, but this configuration was never considered the optimal one after training.

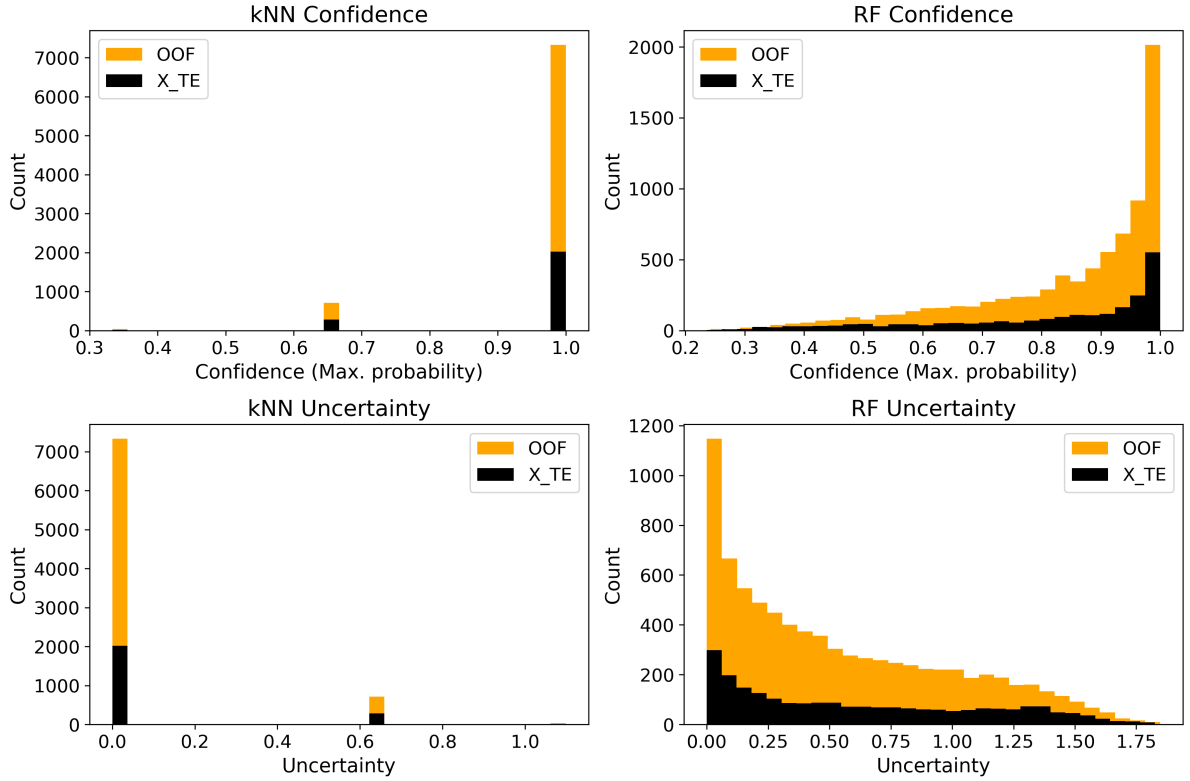


Figure 13: **Confidence and uncertainty for out-of-fold and test data set.**

This figure presents the confidence and uncertainty for the kNN and Random Forest models, both for 5 out-of-fold (OOF) test sets from the original data set, X_TR, and for the test data set, X_TE. For the OOF, repetitions (R) are not shown to facilitate visualization of X_TE results, but the distribution barely changes when we repeat the experiment. One can see how both for kNN and Random Forest in both confidence and uncertainty, distributions between OOF and X_TE are similar. Thus, these results support the hypothesis that the data in the test set might belong to the same distribution as the training data.

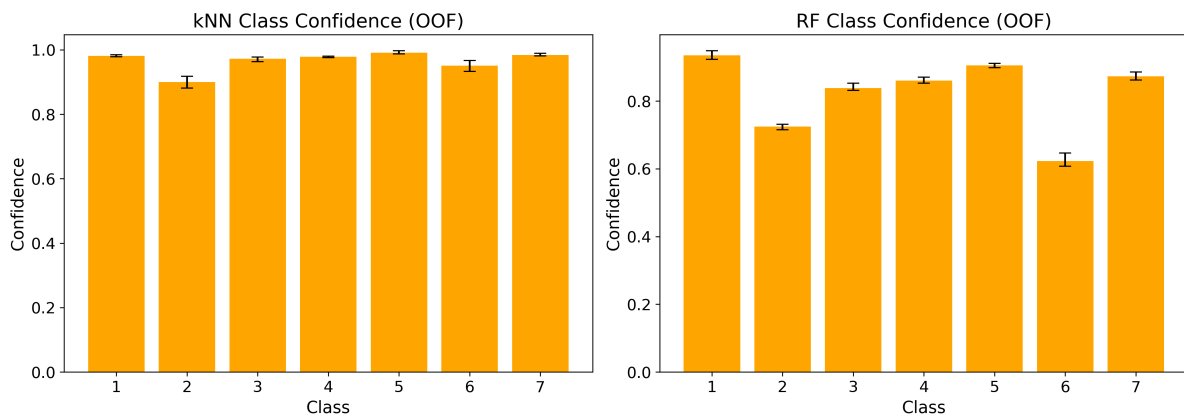


Figure 14: **Confidence per class.** This figure presents the average confidence that the kNN (left) and the Random Forest (right) give when they classify data as a certain class in the out-of-fold tests. We can see how the kNN model presents a high confidence for all classes. The slight decrease in confidence that kNN suffers for class 2 might correspond to the counts in Fig. 13 that did not have the maximum confidence. A decrease in confidence for class 2 is reasonable, because this class has high overlap with class 1. Also, the results show that Random Forest is not so robust, since confidence is, on average, below that in kNN, and per class, there are some prominent decreases that do not appear in the kNN plot. For example, when Random Forest classifies a series as class 2, it also suffers from a decrease in confidence, but the drop is more significant than for kNN. Also, Random Forest's prediction confidence for class 6 is noticeably lower than that in kNN, indicating that the separation of the minority class (see Fig. 16) is not handled as confidently by Random Forest as it is handled by kNN.

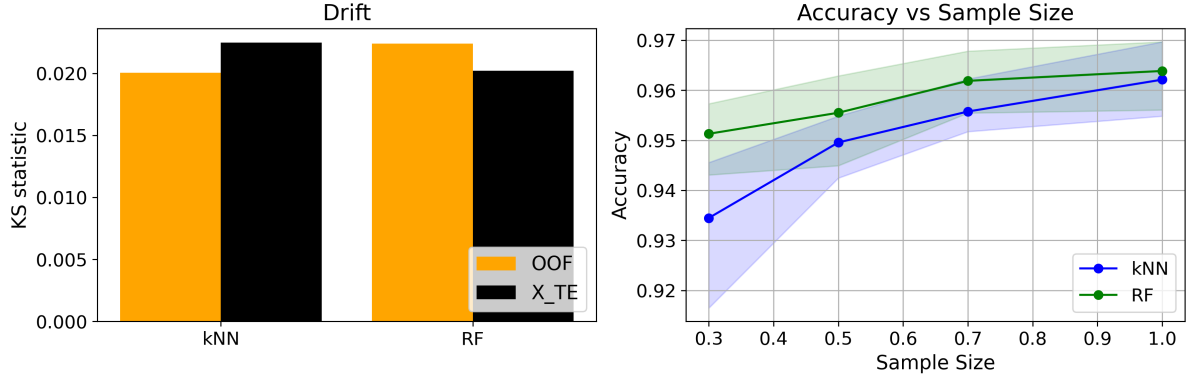


Figure 15: **Drift and accuracies** . This figure shows the average KS statistic found by comparing the training data in each fold with the out-of-fold (OOF) test set with a yellow tone. With a black tone, we present the KS statistic obtained when comparing the distribution from the training data set X_TR and the test data set X_TE. This metric is helpful for determining if drift can be found in the test data set. The KS statistic (and therefore the drift) that we find using Random Forest and using kNN is similar, and so is when comparing the one in out-of-fold and in X_TE. Note, however, that the variations in drift between kNN and Random Forest are given by the fact that we are not using the same number of features per class in kNN and in Random Forest. The most important conclusion here is that, what could appear to be drift by observing that the KS value is around 0.02 for X_TE, is discarded by the fact that the OOF presents the same result. The OOF result should have KS value equal to 0, since in that case we are measuring the drift between series from the training data (in fold) and series from the training data (out of fold). And the reason why it is not 0 is probably because our data set is finite. Thus, these results suggest that there is no evidence of drift, indicating that the distributions in the training set X_TR and in the test set X_TE are likely to be identical. Therefore, for the classification of data from X_TE, we expect a similar accuracy to the one presented in the right plot for sample size = 1 (using the entire X_TR data set).

5 Question 2(a)

During the dimensionality reduction technique followed in this work (what we call the median procedure), we are not taking into account how many series per class we have, since there is no severe class imbalance (see Fig. 16). Recall that to select which features were used for dimensionality reduction, we first selected x features per class (those where series from that class tend to have peaks); but we never took into account the amount of series per class.

Now we consider the case of having a supermajority class (class 1), that will have 1050 data, while the other six classes have 15 data each. I.e. we create a new data set with extreme class imbalance.

To study our pipeline, we train and evaluate as we did in Question 1(a), with results shown in Figure 17. In this figure, we can observe how misleading the overall accuracy is when there is severe class imbalance: when the vast majority of a data set is formed by the majority class, the overall accuracy mainly reflects the classification accuracy for this class, even though the minority classes are being poorly classified. In addition, Figure 17 shows that class 2 is the worst classified of all classes, which is an expected behavior after the previous analysis of Fig. 9 and Fig. 10. Class 1 and class 2 appeared to overlap highly in the feature space; and now that class 1 is the supermajority class, models will ignore class 2 series most of the time when classifying, labeling them as class 1.

The analysis of the results in Fig. 17 is complicated due to the high variability of the results (large error bars), which is given by the fact that only 15 data points are used for each of the minority classes. However, it is clear that LDA performs better than the rest of the models.

The superiority of LDA in this case can be explained by analyzing how this model works. Geometrically, LDA creates ellipsoids with the shape and direction defined by the covariance matrix (computed using all classes), and the centers from these ellipsoids (one per class) are defined by the average data point of each class. Now the shape and direction of the ellipsoids are mainly defined by the data distribution of the supermajority class (class 1). However, for classes really far away from class 1 in the feature space, such as class 5 and 7 (see Fig. 9), the ellipsoids are centered so far away that the shape and direction of the ellipsoids affects less. This explains why LDA classification (especially for these type of classes) is not as low.

With the presence of a supermajority class, kNN and Random Forest, previously leading performance, no longer yield the best results. kNN now drops in performance since local neighborhoods are incomplete, complicating the classification task. Random Forest and logistic regression will ignore the minority class for the most part, since the penalization received is the same when misclassifying the majority or the minority class.

Note that although LDA performs better than the rest of the models, they all yield low performance for the minority classes when we compare the results to the case of having no severe class imbalance (Fig. 3). For class 6, performance is still low across all models due to its overlap with multiple classes, as we saw previously in the report (Fig. 9).

To improve the classification of series from the minority classes, we first try weighting. The results for this approach are presented in Fig. 18, with a clear improvement in performance across all classes. Only logistic regression and Random Forest are presented since they are the only models for which weighting can be applied without sophisticated methods.

We then also try oversampling to improve the classification performance for minority classes. In this case, for each minority class, we duplicate its data (randomly) until the number of elements in the minority class reaches the number of data in the majority class. This forces our models to take into account series from minority classes as much as it considers majority-class series. With this approach, the classification performance across all classes increases noticeably compared to the results in Fig. 17. See Fig. 19 for the oversampling results.

5.1 Conclusions

- Since we want to study the case of having a data set with a supermajority class, we will create a data set using 1050 series from class 1, and 15 series for each of the other classes.
- Using the same four models as in Question 1(a), we obtained poor accuracy when classifying the minority classes; although LDA appeared to be more robust than the rest of the models, especially for the classes more separated in feature space from class 1.
- Given these results, we trained our models using weighting, to highly penalize a misclassification from a minority-class series. With this approach we try to avoid models from ignoring the minority classes. Results show that the classification performance for series from the minority classes increases significantly (Fig. 18). Even for class 2, highly overlapping with class 1, the results improve. This might be given because class 2 is more concentrated in space than class 1 (see Fig. 9), and when we weight class 2, the region where these two classes overlap starts to be 'dominated' by class 2 series. This argument (combined with the fact that weighting reduces the 'importance' that models place in correctly classifying class 1) would also explain the decrease in classification performance for class 1.
- The effect of oversampling was also studied with Fig. 19 with performance being similar to the case of weighting.

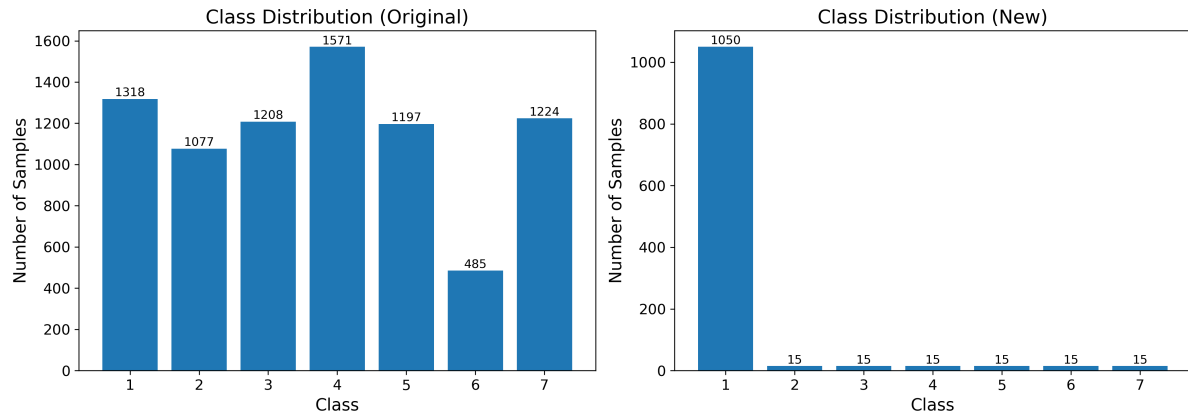


Figure 16: **Number of series per class.** This figure presents the number of series per class in the original data set (left) and in the new data set (right). The new data set studies severe class imbalance keeping 1050 series for class 1 (supermajority class), while we only keep 15 samples from each of the other classes (minority classes).

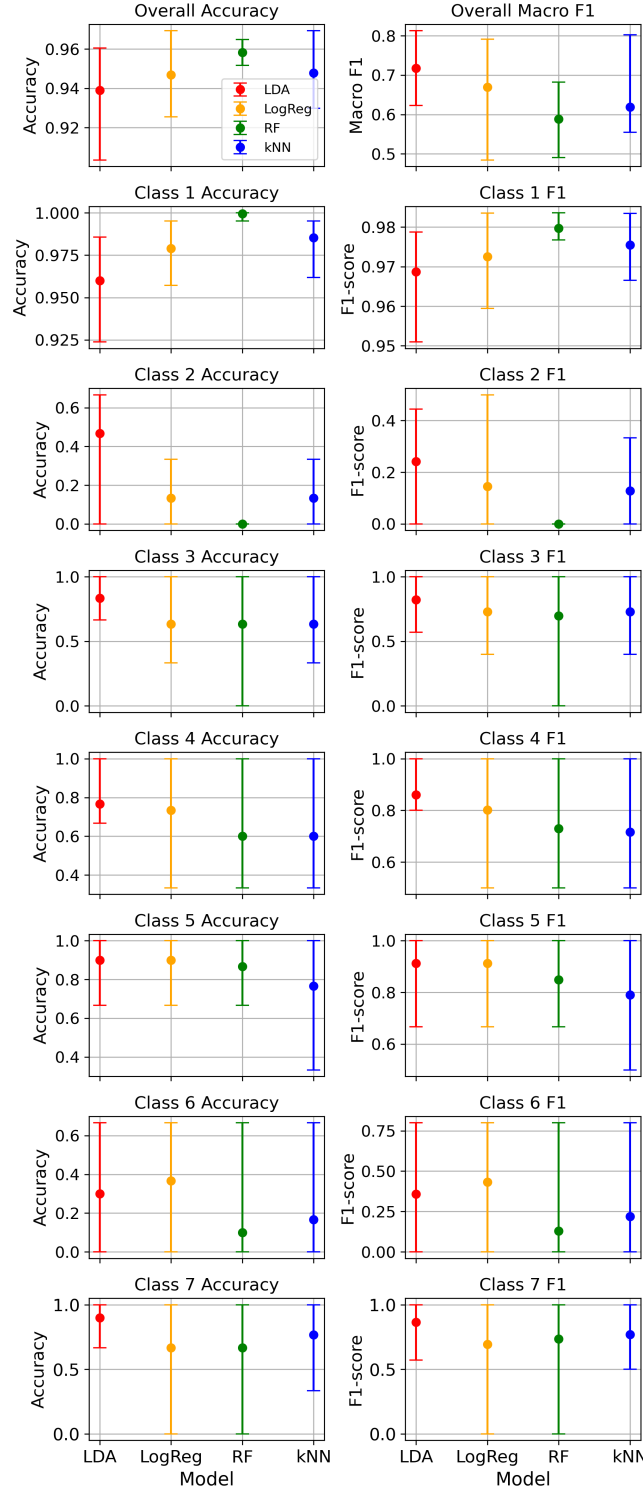


Figure 17: **Case of supermajority class - Overall and class-specific performance.** This figure presents overall accuracy and F1 (first row), and the class-specific accuracies and F1 scores, when class 1 is a supermajority class. The overall accuracy is misleading since it essentially only captures the classification accuracy of series from class 1. When analyzing the class-specific accuracies, we can see that the minority classes are poorly classified. Class 2 is the worst classified, due to a high overlap with class 1 in the feature space; and class 6 is also poorly classified due to overlap with other classes (see Fig. 9).

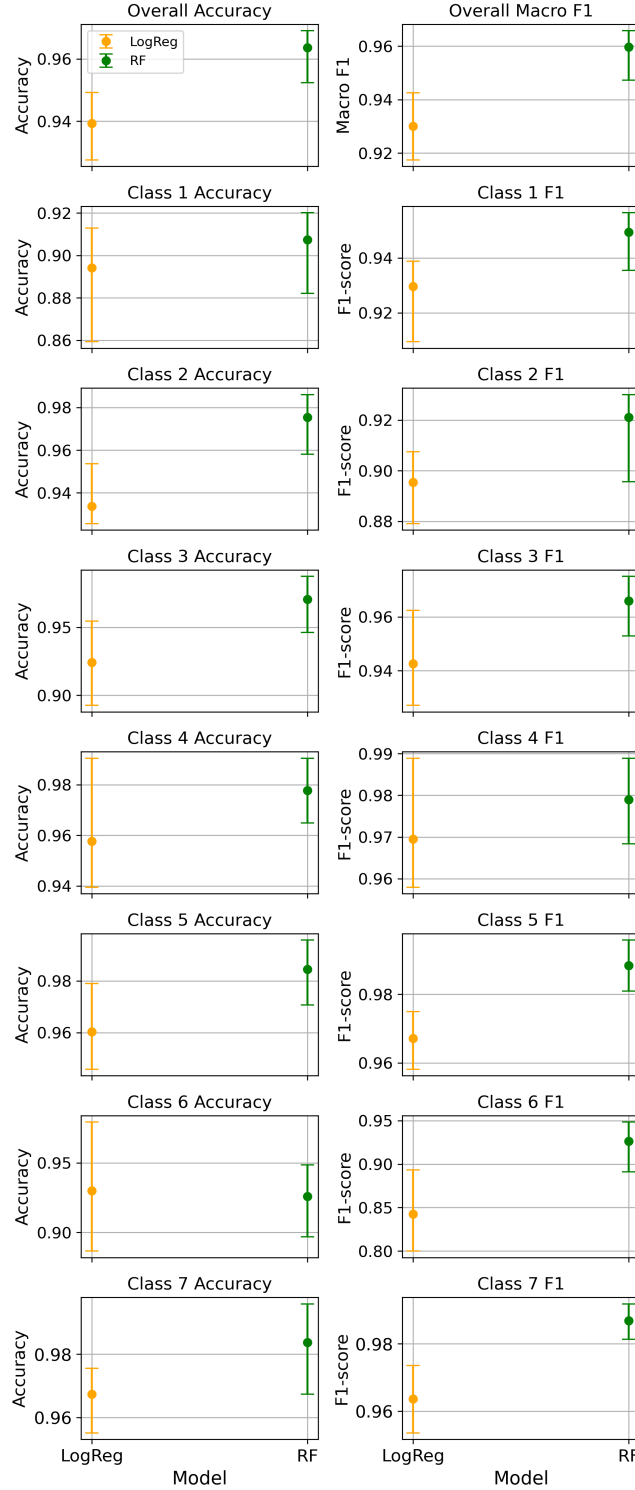


Figure 18: **Case of supermajority class - Overall and class-specific performance using weighting.** This figure presents overall accuracy and F1 (first row), and the class-specific accuracies and F1 scores, when class 1 is a supermajority class and weights are used to train the models. Only logistic regression and Random Forest are presented, since they are the only models accepting weighting without sophisticated procedures. With weighting, results improve noticeably for both models and all classes. This is expected, since now models receive a high penalty when they incorrectly classify a series from a minority class while training; and thus, do not ignore the minority classes as vastly happened without weights (see Fig. 17).

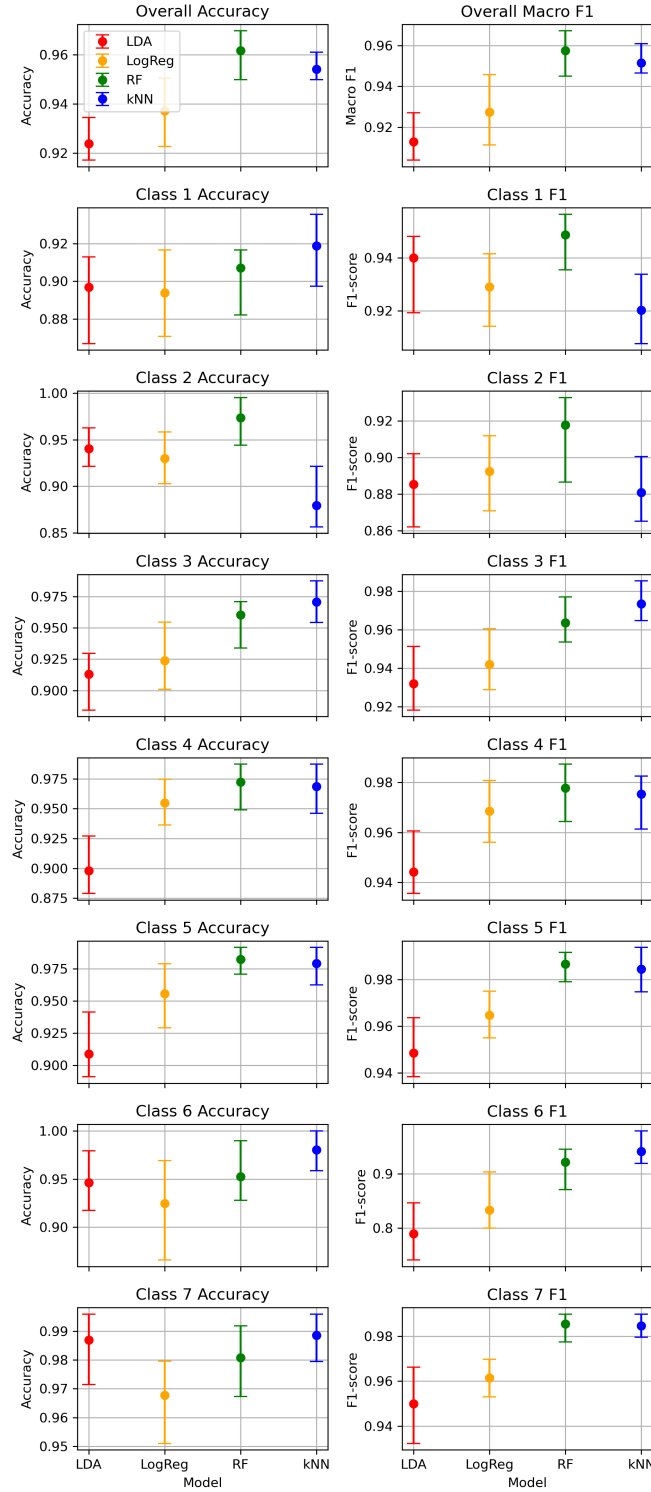


Figure 19: **Case of supermajority class - Overall and class-specific performance using oversampling.** This figure presents overall accuracy and F1 (first row), and the class-specific accuracies and F1 scores, when class 1 is a supermajority class and oversampling is used to train the models. Now, minority classes data points are duplicated (randomly) until reaching the same number of data present in the majority class. With oversampling, results improve noticeably for all models and classes. This is expected, since the increase in density for the minority classes will prevent models from ignoring the minority classes when training.

6 Question 2(b)

In this case, we want to handle data classification from the new dataset (with a supermajority class) using a different approach to weighting or oversampling, as used in Question 2(a). Here, we divide the classification problem in two separate tasks: first, we train a model to separate the majority class from all the minority classes; and secondly, we train new models to separate between minority classes.

Here we will not combine all models that could be used for the first classification task (majority-vs-rest) with all the models that could be used for the second classification problem (multiclass problem for minority classes). Instead, we first study how different models perform the first task, and combine the best performing one with the different models from the second task.

For the majority-vs-rest problem, we train our four models optimizing for the models' hyperparameters and number of features selected per class. Then we analyze how well our models perform overall, measuring accuracy, balanced accuracy, per-class accuracy, and per-class F1 score (Fig. 20). The balanced accuracy shows that our models are highly capable of distinguishing between these two groups (majority vs rest), especially LDA and Random Forest. Note, however, that results for the group 'rest' are far away from the classification performance that we had when there was no severe class imbalance. See for example Fig. 3, where nearly all classes were classified with more than 90% accuracy and F1 score. Even though we are grouping all minority classes, making the problem a 2-class task, there is still a high class imbalance: while class 1 has 1050 data, the 'rest' group only has 90 series (15×6). Therefore, even though the 'rest' group is now more considered by models than each individual minority class was in Question 2(a), the majority-vs-rest problem is still highly imbalanced.

For the full pipeline, we train a Random Forest for the majority-vs-rest problem, and we also train four models for the multiclass problem of minority classes. For classification, a series is firstly classified as 'majority class' or 'rest' by the first classifier; and if 'rest' is predicted, the second classifier will assign a minority class to it.

The results for the full pipeline (Fig. 21) show that minority classes are still poorly classified in most cases.

These results are reasonable: even though our first classifier helps in distinguishing the majority class from the rest, performance for this task is already far away from perfection. However, another factor is also responsible for the low classification performance of minority classes: the amount of data. If the majority-vs-rest classifier worked perfectly, we would expect, on average, to have no class imbalance for the second stage, since we used 15 data for each minority class. However, this lack of imbalance will most likely never occur because, for example, the first classifier will probably label most of class 2 series as class 1 due to high overlap in feature space (Fig. 9). Also, 15 data points per class is not enough for models to learn the real distribution of data from each class in the feature space, which complicates the second stage's classification even if the first stage worked perfectly.

Note, however, that the most separable classes (class 5 and class 7) are decently classified when using LDA and Random Forest as the second stage models. This could be given by the significant separation that exists between data distributions from these classes and that from class 1 (see Fig. 9). This separation probably makes the majority-vs-rest model classify most series from these classes as part of 'rest', which is already a major step. Then, since they are also highly separated from other minority classes, the second stage classifies them properly.

The kNN model will not be particularly useful in this case, given that we have few data points per minority class. Depending on how the training data lie in the feature space for each class, the optimal k can change noticeably, making it impossible for the model to properly learn which is the optimal k for the entire data set (real data distributions).

6.1 Conclusions

- In this case we follow a 2-stage classification procedure. First, we combine all the data from the minority classes in a 'rest' group, and we train different models for the majority-vs-rest problem. The Random Forest was one of the best performing ones, so it was kept to act as the majority-vs-rest classifier. For the second stage, we train different models to separate between the minority classes. In each fold of the nested CV, we use the training data to train both the first model and the second models.
- Using this procedure, we can see that distinguishing the majority class from the 'rest' group is already a challenging task. This is because for this problem, there is still a significant class imbalance: 1050 data for the majority class, and 90 data for the 'rest' group.
- When we analyze the results for the full pipeline, minority classes are still poorly classified. Added to the problem of separating between the majority class and the 'rest' group, we find the few data that we have for the minority classes. With so few data per class, our models will not be able to properly learn the class distributions, which highly complicates the multiclass problem for minority classes.
- Despite of all these difficulties, class 5 and class 7 are decently classified. This probably occurs because their data distributions in the feature space are highly separated from class 1, which increases the probability that they are classified as 'rest'. In addition, they are well separated from the other minority classes, which helps the second stage model classify them properly.

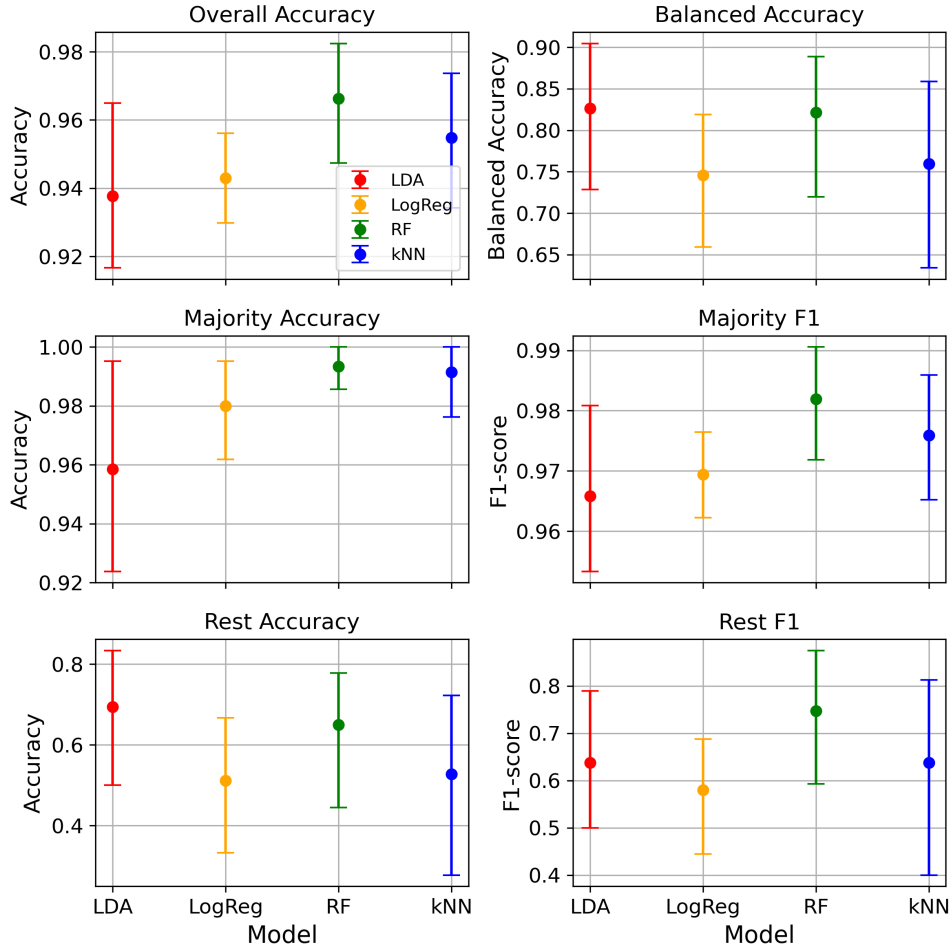


Figure 20: **Classification performance of majority-vs-rest classifier.** This figure shows the overall accuracy (top left) and the balanced accuracy (top right) for the majority-vs-rest problem. Classification performance for the majority class is presented in the middle row, while classification performance for the 'rest' group is presented in the bottom row. The overall accuracy is near 1 because the data from the majority class are properly classified; and since the majority class dominates the data set, the overall accuracy will reflect the accuracy of class 1 classification (see middle row). The balanced accuracy is a metric that takes more into consideration the 'rest' group than the overall accuracy does, and presents lower values due to a worse classification of series from this group. The balanced accuracy show that LDA and Random Forest are the best models for the majority-vs-rest problem. Note, also, that the F1 for class 1 is not low because there are much more values from the majority class than from the 'rest' group, so classifying minority-class series as class 1 will not be highly taken into account in this metric.

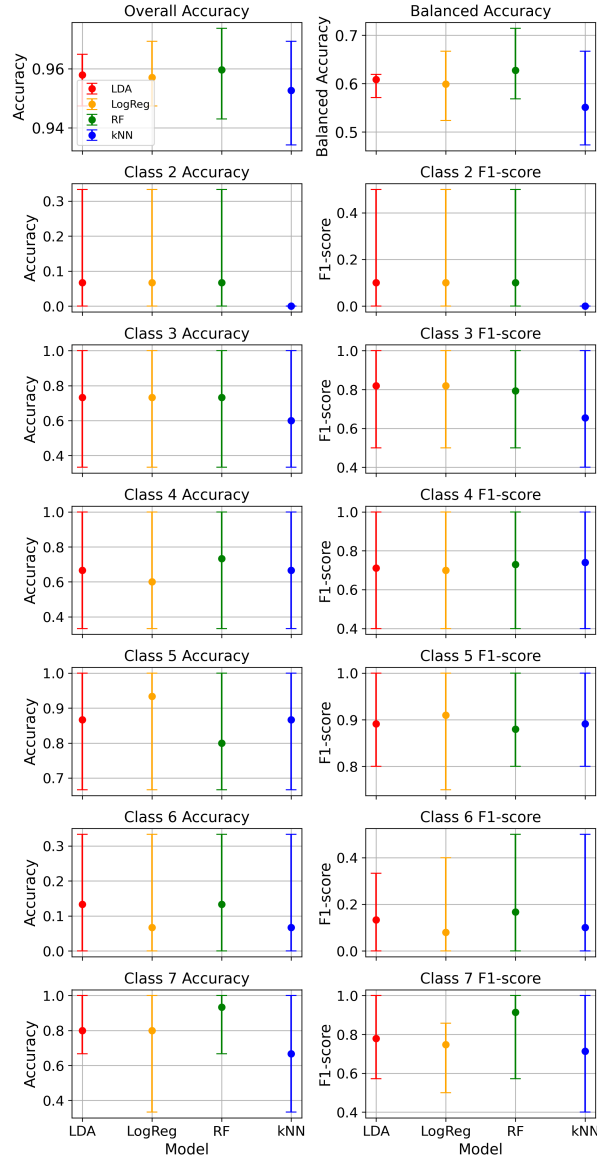


Figure 21: **Classification performance for the full pipeline.** This figure shows the overall accuracy (top left) and the balanced accuracy (top right) for the classification problem when considering the full pipeline. Per-class accuracies also refer to the accuracy over the full pipeline, not the classification performance of the second classifiers. Classification performance over class 1 was not shown to avoid redundancy, since results will follow the same distribution as those in Fig. 20. Results show that models are highly irregular, which is reasonable: now that there are few data points per class, it is complicated for models to capture the real distribution of data points from each class, making the results highly irregular (large error bars). Class 2 is still poorly classified due to high overlap with class 1 (see Fig. 9); and even though the first classifier might help in distinguishing slightly better between class 1 and the 'rest', making classification of class 2 slightly worse, it is not sufficient to distinguish properly. Class 6 still yields low performance due to high overlap with other classes.